

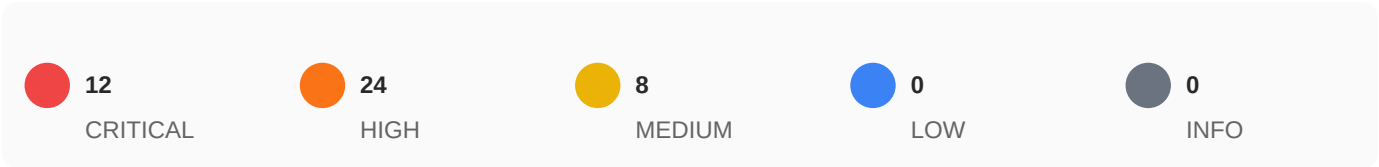
VibeGuard Security Report

Target: c:\Users\mahme\OneDrive\Desktop\CodeShield AI
Generated: 6/13/2026, 8:49:21 PM | v1.4.0

0

Grade: F

Files Scanned: 38
Lines of Code: 9,758
Duration: 5.8s
Total Issues: 44



Vulnerability Findings

HIGH

Path Traversal

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\vscode-extension\src\pdf-exporter.ts:194

The code is vulnerable to path traversal attacks because it directly uses user-provided input (outputPath) to write a file without proper validation. An attacker could exploit this by providing a path that traverses outside the intended directory, potentially allowing them to write files in arbitrary locations on the system.

Remediation: To fix this vulnerability, validate and sanitize the outputPath to ensure it does not contain any traversal sequences ('..' or '..\') and that it is within an intended directory. Consider using a library that provides secure path handling.

HIGH

Path Traversal

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\cli\src\pdfReport.ts:82

The code is vulnerable to path traversal attacks because the outputPath parameter is derived from user input without validation. An attacker could use a malicious path like ../../etc/passwd to read arbitrary files.

Remediation: Validate the outputPath parameter to prevent path traversal attacks. Use a library like path to normalize the path and ensure it's within the expected directory.

MEDIUM

Path Traversal

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\cli\src\index.ts:25

path.join() called with a non-literal argument — verify that path segments are validated.

Remediation: After path.join(), verify the result is within your intended base directory using path.resolve().

HIGH

Path Traversal

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\cli\src\index.ts:243

fs.writeFileSync() called with a non-literal path — potential path traversal.

Remediation: Validate and sanitize all path inputs. Use path.resolve() and verify the result stays within an allowed base directory:

```
const safePath = path.resolve(baseDir, userInput);  
if (!safePath.startsWith(baseDir)) throw new Error("Access denied")
```

HIGH

Path Traversal

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\cli\src\index.ts:361

fs.writeFileSync() called with a non-literal path — potential path traversal.

Remediation: Validate and sanitize all path inputs. Use path.resolve() and verify the result stays within an allowed base directory:

```
const safePath = path.resolve(baseDir, userInput);  
if (!safePath.startsWith(baseDir)) throw new Error("Access denied")
```

MEDIUM

Path Traversal

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\cli\src\index.ts:374

path.join() called with a non-literal argument — verify that path segments are validated.

Remediation: After path.join(), verify the result is within your intended base directory using path.resolve().

HIGH**Path Traversal**

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\cli\src\index.ts:395
fs.writeFileSync() called with a non-literal path — potential path traversal.

Remediation: Validate and sanitize all path inputs. Use path.resolve() and verify the result stays within an allowed base directory:

```
const safePath = path.resolve(baseDir, userInput);  
if (!safePath.startsWith(baseDir)) throw new Error("Access denied")
```

MEDIUM**Path Traversal**

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\vscode-extension\src\extension.ts:533
The path.join() function is used with dynamic segments that may come from user-controlled sources, introducing a potential path traversal vulnerability.

Remediation: Validate and sanitize user input before passing it to path.join() to prevent path traversal attacks.

HIGH**Path Traversal**

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\vscode-extension\src\extension.ts:549
The path is derived from user input without validation, allowing attackers to use ../../etc/passwd to read arbitrary files.

Remediation: Validate and sanitize user input before using it to construct a file path to prevent path traversal attacks.

HIGH**Path Traversal**

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\vscode-extension\src\extension.ts:550
The path is derived from user input without validation, allowing attackers to use ../../etc/passwd to read arbitrary files.

Remediation: Validate and sanitize user input before using it to construct a file path to prevent path traversal attacks.

HIGH**Cross-Site Scripting (XSS)**

VG-XSS-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\frontend\js\app.js:456
Unsafe assignment to "innerHTML" may allow XSS if the value contains user-controlled input.

Remediation: Use.textContent instead of innerHTML for plain text. For HTML, sanitize using DOMPurify: element.innerHTML = DOMPurify.sanitize(userInput)

HIGH

Cross-Site Scripting (XSS)

VG-XSS-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\frontend\js\app.js:562

Unsafe assignment to "innerHTML" may allow XSS if the value contains user-controlled input.

Remediation: Use `textContent` instead of `innerHTML` for plain text. For HTML, sanitize using `DOMPurify`: `element.innerHTML = DOMPurify.sanitize(userInput)`

HIGH

Cross-Site Scripting (XSS)

VG-XSS-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\frontend\js\app.js:584

Unsafe assignment to "innerHTML" may allow XSS if the value contains user-controlled input.

Remediation: Use `textContent` instead of `innerHTML` for plain text. For HTML, sanitize using `DOMPurify`: `element.innerHTML = DOMPurify.sanitize(userInput)`

HIGH

Cross-Site Scripting (XSS)

VG-XSS-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\frontend\js\app.js:875

Unsafe assignment to "innerHTML" may allow XSS if the value contains user-controlled input.

Remediation: Use `textContent` instead of `innerHTML` for plain text. For HTML, sanitize using `DOMPurify`: `element.innerHTML = DOMPurify.sanitize(userInput)`

HIGH

Cross-Site Scripting (XSS)

VG-XSS-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\frontend\js\app.js:877

Unsafe assignment to "innerHTML" may allow XSS if the value contains user-controlled input.

Remediation: Use `textContent` instead of `innerHTML` for plain text. For HTML, sanitize using `DOMPurify`: `element.innerHTML = DOMPurify.sanitize(userInput)`

HIGH

Cross-Site Scripting (XSS)

VG-XSS-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\frontend\js\app.js:889

Unsafe assignment to "innerHTML" may allow XSS if the value contains user-controlled input.

Remediation: Use `textContent` instead of `innerHTML` for plain text. For HTML, sanitize using `DOMPurify`: `element.innerHTML = DOMPurify.sanitize(userInput)`

HIGH**Cross-Site Scripting (XSS)**

VG-XSS-001

`c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\frontend\js\app.js:933`

Unsafe assignment to "innerHTML" may allow XSS if the value contains user-controlled input.

Remediation: Use `textContent` instead of `innerHTML` for plain text. For HTML, sanitize using `DOMPurify`: `element.innerHTML = DOMPurify.sanitize(userInput)`

MEDIUM**Path Traversal**

VG-PATH-001

`c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\vscode-extension\build.js:16`

`path.join()` called with a non-literal argument — verify that path segments are validated.

Remediation: After `path.join()`, verify the result is within your intended base directory using `path.resolve()`.

MEDIUM**Path Traversal**

VG-PATH-001

`c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\vscode-extension\build.js:17`

`path.join()` called with a non-literal argument — verify that path segments are validated.

Remediation: After `path.join()`, verify the result is within your intended base directory using `path.resolve()`.

MEDIUM**Path Traversal**

VG-PATH-001

`c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\vscode-extension\build.js:18`

`path.join()` called with a non-literal argument — verify that path segments are validated.

Remediation: After `path.join()`, verify the result is within your intended base directory using `path.resolve()`.

HIGH**Path Traversal**

VG-PATH-001

`c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\core\src\scanner.ts:93`

`fs.statSync()` called with a non-literal path — potential path traversal.

Remediation: Validate and sanitize all path inputs. Use `path.resolve()` and verify the result stays within an allowed base directory:

```
const safePath = path.resolve(baseDir, userInput);
if (!safePath.startsWith(baseDir)) throw new Error("Access denied")
```

HIGH**Path Traversal**

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\core\src\scanner.ts:142
fs.statSync() called with a non-literal path — potential path traversal.

Remediation: Validate and sanitize all path inputs. Use path.resolve() and verify the result stays within an allowed base directory:

```
const safePath = path.resolve(baseDir, userInput);  
if (!safePath.startsWith(baseDir)) throw new Error("Access denied")
```

HIGH**Path Traversal**

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\core\src\scanner.ts:148
fs.readFileSync() called with a non-literal path — potential path traversal.

Remediation: Validate and sanitize all path inputs. Use path.resolve() and verify the result stays within an allowed base directory:

```
const safePath = path.resolve(baseDir, userInput);  
if (!safePath.startsWith(baseDir)) throw new Error("Access denied")
```

HIGH**Path Traversal**

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\core\src\scanner.ts:275
fs.statSync() called with a non-literal path — potential path traversal.

Remediation: Validate and sanitize all path inputs. Use path.resolve() and verify the result stays within an allowed base directory:

```
const safePath = path.resolve(baseDir, userInput);  
if (!safePath.startsWith(baseDir)) throw new Error("Access denied")
```

HIGH**Path Traversal**

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\core\src\sca-engine.ts:75
fs.readFileSync() called with a non-literal path — potential path traversal.

Remediation: Validate and sanitize all path inputs. Use path.resolve() and verify the result stays within an allowed base directory:

```
const safePath = path.resolve(baseDir, userInput);  
if (!safePath.startsWith(baseDir)) throw new Error("Access denied")
```

HIGH**Path Traversal**

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\core\src\sca-engine.ts:243
fs.readdirSync() called with a non-literal path — potential path traversal.

Remediation: Validate and sanitize all path inputs. Use path.resolve() and verify the result stays within an allowed base directory:

```
const safePath = path.resolve(baseDir, userInput);  
if (!safePath.startsWith(baseDir)) throw new Error("Access denied")
```

MEDIUM**Path Traversal**

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\core\src\sca-engine.ts:249
path.join() called with a non-literal argument — verify that path segments are validated.

Remediation: After path.join(), verify the result is within your intended base directory using path.resolve().

HIGH**Path Traversal**

VG-PATH-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\vibeguard\core\src\sca-engine.ts:263
fs.statSync() called with a non-literal path — potential path traversal.

Remediation: Validate and sanitize all path inputs. Use path.resolve() and verify the result stays within an allowed base directory:

```
const safePath = path.resolve(baseDir, userInput);  
if (!safePath.startsWith(baseDir)) throw new Error("Access denied")
```

CRITICAL**Python eval/exec Injection**

PY-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\rules.py:108
eval()/exec() used with dynamic input on line 108 — potential code injection.

Remediation: Avoid eval()/exec() entirely. Use ast.literal_eval() for safe evaluation of literals, or refactor to avoid dynamic code execution.

CRITICAL

Python eval/exec Injection

PY-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\rules.py:111
eval()/exec() used with dynamic input on line 111 — potential code injection.

Remediation: Avoid eval()/exec() entirely. Use ast.literal_eval() for safe evaluation of literals, or refactor to avoid dynamic code execution.

CRITICAL

Python eval/exec Injection

PY-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\rules.py:112
eval()/exec() used with dynamic input on line 112 — potential code injection.

Remediation: Avoid eval()/exec() entirely. Use ast.literal_eval() for safe evaluation of literals, or refactor to avoid dynamic code execution.

CRITICAL

Python eval/exec Injection

PY-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\rules.py:113
eval()/exec() used with dynamic input on line 113 — potential code injection.

Remediation: Avoid eval()/exec() entirely. Use ast.literal_eval() for safe evaluation of literals, or refactor to avoid dynamic code execution.

CRITICAL

Python eval/exec Injection

PY-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\rules.py:116
eval()/exec() used with dynamic input on line 116 — potential code injection.

Remediation: Avoid eval()/exec() entirely. Use ast.literal_eval() for safe evaluation of literals, or refactor to avoid dynamic code execution.

CRITICAL

Python Command Injection

PY-002

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\rules.py:127
Shell command execution detected at line 127. If user input is included, this is exploitable.

Remediation: Use subprocess.run() with a list of arguments (shell=False). Example: subprocess.run(["ls", "-la"], shell=False)

HIGH**Python Insecure Deserialization (pickle)**

PY-003

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\rules.py:172
pickle.load/loads() at line 172 — deserializing untrusted data can execute arbitrary code.

Remediation: Use JSON, MessagePack, or other safe serialization formats. If pickle is required, use HMAC signatures to verify data integrity before deserializing.

HIGH**Python Hardcoded Secret**

PY-005

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\rules.py:84
Possible hardcoded credential at line 84.

Remediation: Use environment variables: `import os; password = os.environ.get("PASSWORD")`. Consider python-dotenv or a secrets manager.

HIGH**Python Hardcoded Secret**

PY-005

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\rules.py:98
Possible hardcoded credential at line 98.

Remediation: Use environment variables: `import os; password = os.environ.get("PASSWORD")`. Consider python-dotenv or a secrets manager.

MEDIUM**Python Insecure Random**

PY-007

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\rules.py:157
random module used at line 157. Not suitable for cryptographic operations.

Remediation: Use secrets module: `import secrets; token = secrets.token_hex(32)`. Or use `os.urandom()` for raw bytes.

CRITICAL**Python eval/exec Injection**

PY-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\last_analyzer.py:23
eval()/exec() used with dynamic input on line 23 — potential code injection.

Remediation: Avoid eval()/exec() entirely. Use `ast.literal_eval()` for safe evaluation of literals, or refactor to avoid dynamic code execution.

CRITICAL

Python eval/exec Injection

PY-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\last_analyzer.py:24
eval()/exec() used with dynamic input on line 24 — potential code injection.

Remediation: Avoid eval()/exec() entirely. Use ast.literal_eval() for safe evaluation of literals, or refactor to avoid dynamic code execution.

CRITICAL

Python eval/exec Injection

PY-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\last_analyzer.py:179
eval()/exec() used with dynamic input on line 179 — potential code injection.

Remediation: Avoid eval()/exec() entirely. Use ast.literal_eval() for safe evaluation of literals, or refactor to avoid dynamic code execution.

CRITICAL

Python eval/exec Injection

PY-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\last_analyzer.py:181
eval()/exec() used with dynamic input on line 181 — potential code injection.

Remediation: Avoid eval()/exec() entirely. Use ast.literal_eval() for safe evaluation of literals, or refactor to avoid dynamic code execution.

CRITICAL

Python eval/exec Injection

PY-001

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\last_analyzer.py:188
eval()/exec() used with dynamic input on line 188 — potential code injection.

Remediation: Avoid eval()/exec() entirely. Use ast.literal_eval() for safe evaluation of literals, or refactor to avoid dynamic code execution.

CRITICAL

Python Command Injection

PY-002

c:\Users\mahme\OneDrive\Desktop\CodeShield AI\webapp\backend\app\last_analyzer.py:31
Shell command execution detected at line 31. If user input is included, this is exploitable.

Remediation: Use subprocess.run() with a list of arguments (shell=False). Example: subprocess.run(["ls", "-la"], shell=False)